



Obiectiv general

Scopul proiectului este de a realiza o **aplicație cloud-native** folosind elementele învățate în cadrul laboratoarelor de Cloud Computing.

Deoarece tehnologiile moderne de orchestrare, precum **Kubernetes**, au sens deplin în contextul aplicațiilor bazate pe microservicii, codul vostru trebuie să respecte **filozofia microserviciilor**. Astfel, aplicația va fi alcătuită din **componente modulare și autonome**, care colaborează pentru a oferi funcționalitatea completă a sistemului. Fiecare microserviciu va rula independent și va avea propriile resurse, configurări și cicluri de viață, reprezentate în Kubernetes prin **Deployment-uri** și **Servicii**. Comunicarea dintre microservicii se poate realiza în mod **sincron** (de exemplu prin cereri HTTP) sau **asincron** (prin cozi de mesaje ori alte mecanisme de schimb de date).

Nu există restricții privind **limbajul de programare** sau tehnologiile folosite. Alegerea este la latitudinea echipei, atât timp cât soluția finală respectă principiile modularității și poate fi orchestrată într-un cluster Kubernetes.

Proiectul trebuie să demonstreze o aplicare practică a noțiunilor din laboratoarele 1-5:

- containerizare și imagini Docker (Lab 1)
- cluster Kubernetes, pods și servicii (Lab 2-3)
- gestionarea aplicațiilor cu Helm (Lab 4)
- monitorizarea și observabilitatea clusterului (Lab 5).

Cerințe funcționale minime

Proiectul trebuie să includă cel puțin cinci microservicii distincte, containerizate și orchestrate prin Kubernetes, astfel:

1. Microserviciu de autentificare și autorizare
 - a. gestionează autentificarea utilizatorilor și controlul accesului
 - b. poate folosi orice mecanism intern de validare (fără constrângeri de protocol sau limbaj)
 - c. trebuie să fie un serviciu separat, containerizat și integrat cu restul aplicației.
2. Microserviciu de business logic
 - a. reprezintă componenta principală a aplicației, responsabilă de funcționalitatea de bază
 - b. interacționează cu celelalte microservicii (autentificare, bază de date, etc.) conform arhitecturii definite de echipă
 - c. poate comunica prin orice mecanism ales (HTTP, RPC, etc.).
3. Microserviciu de tip bază de date
 - a. oferă stocarea persistentă a datelor aplicației
 - b. poate fi MySQL, PostgreSQL, MongoDB, Redis sau un alt sistem echivalent
 - c. trebuie configurat cu un PersistentVolumeClaim și expus printr-un Service intern.
4. Microserviciu de gestiune a bazei de date
 - a. interfață grafică pentru administrarea și inspecția datelor (Adminer, phpMyAdmin, DBeaver, etc.)

- b. trebuie conectat la microserviciul de bază de date și să permită interogarea directă a acestuia.
5. Microserviciu utilitar grafic de gestiune a clusterului
- a. instrument vizual precum Portainer, Rancher sau Yacht
 - b. rulează în Kubernetes și oferă o imagine asupra resurselor aplicației (pods, servicii, volume).

Integrare și orchestrare

1. Containerizare completă (Lab 1)
 - a. fiecare microserviciu scris de voi are propriul *Dockerfile*
 - b. imaginile pot fi construite local sau extrase dintr-un registru public.
2. Configurare Kubernetes (Lab 2-3)
 - a. fiecare microserviciu are un Deployment și un Service asociat
 - b. serviciile trebuie configurate pentru comunicare internă și acces extern (NodePort sau Ingress)
 - c. baza de date trebuie să utilizeze un volum persistent (PersistentVolumeClaim).
3. Gestionare prin Helm (Lab 4)
 - a. se va crea un chart Helm complet care include toate componentele aplicației
 - b. parametrii (imagini, porturi, volume, replici etc.) vor fi centralizați în *values.yaml*
 - c. se vor demonstra instalarea, actualizarea și deinstalarea aplicației prin comenzi Helm.
4. Monitorizare (Lab 5)
 - a. configurarea unui sistem de monitorizare folosind Metrics Server, Prometheus și Grafana
 - b. vizualizarea de metrice pentru poduri, noduri și aplicații
 - c. crearea unui dashboard Grafana care afișează resursele aplicației (CPU, memorie, uptime, etc.).

Livrabile

1. Arhiva proiectului, care va conține:
 - a. codul sursă al microserviciilor dezvoltate
 - b. fișierele *Dockerfile* pentru toate componentele
 - c. resursele Kubernetes (Deployment, Service, PVC) și chart-urile Helm (*Chart.yaml*, *values.yaml*, *templates/*)
 - d. capturi de ecran din Adminer/phpMyAdmin, Portainer și Grafana
 - e. fișier *README.md* cu instrucțiuni clare pentru:
 - i. construirea imaginilor
 - ii. instalarea aplicației prin Helm
 - iii. accesarea fiecărui microserviciu și a dashboard-ului Grafana.
2. Demonstrarea practică (15 minute, la final de semestru)
 - a. rularea completă a aplicației în cluster local (Kind, Minikube)
 - b. accesarea tuturor microserviciilor: autentificare, business logic, DB, Adminer, Portainer, Grafana
 - c. prezentarea dashboard-ului Grafana și a resurselor monitorizate.

Barem de evaluare

Criteriu	Punctaj
Microserviciu de autentificare funcțional și integrat	0.8 p
Microserviciu de business logic funcțional și integrat	0.8 p
Microserviciu de bază de date + stocare persistentă	0.6 p
Microserviciu de gestiune DB (Adminer/phpMyAdmin)	0.3 p
Microserviciu utilitar de cluster (Portainer/Rancher/Yacht)	0.3 p
Helm chart complet și funcțional (instalare, upgrade, remove)	0.6 p
Sistem de monitorizare (Prometheus + Grafana) + dashboard aplicație	0.6 p
Total	4.0 p

Exemple de proiecte posibile

- Sistem de rezervări de resurse
 - serviciu pentru rezervări (business logic)
 - PostgreSQL + Adminer
- Aplicație de management al sarcinilor (task manager)
 - serviciu pentru utilizatori și sarcini
 - MySQL + phpMyAdmin
- Platformă de colectare date senzori IoT
 - serviciu de colectare date
 - MongoDB + Adminer
- Catalog de produse (mini e-commerce)
 - serviciu de utilizatori și produse
 - MySQL + Adminer
- Aplicație de evidență a studenților
 - serviciu pentru note și cursuri
 - PostgreSQL + phpMyAdmin.

Observații finale

- Toate componentele trebuie să ruleze local (pe un cluster Kind sau Minikube) sau online (într-un cluster Kubernetes)
- Nu este permisă conectarea directă între containere în afara Kubernetes
- Bonus (max. 0.5 p):
 - integrare cu un sistem de logging (ex., Loki)
 - adăugarea unui mecanism simplu de autoscaling.